

CSE232 시스템프로그래밍(캡스톤디자인)

3주차 두 번째 수업

저수준 파일 입출력 견고한 반복문과 파일 복사

경민기

오늘의 구성 · 120분

1부 지난 시간 복습 · 15분

2부 read와 오류 처리 · 20분

3부 write_all 구현 · 25분

휴식 · 10분

4부 파일 복사와 테스트 · 40분

퀴즈와 제출 · 10분

1부

지난 시간 복습

파일 프로그램 4개의 반환값과 동작을 확인합니다.

파일 프로그램 4개

file_create: 새로운 빈 파일 생성

file_write: 덮어쓰기 또는 추가 기록

file_read: EOF까지 읽고 출력

file_seek: 바이트 위치를 이동해 수정

반환값 복습

open: 성공한 FD는 0 이상

read / write: 실제 처리한 바이트 수

오류를 나타내는 -1은 ssize_t로 받는다.

close: 성공 0, 실패 -1

결과 예측

```
/* ABCDEFGHIJ 파일, O_RDWR */  
lseek(fd, 3, SEEK_SET);  
read(fd, buf, 2);  
/* 현재 위치는? */  
lseek(fd, 3, SEEK_SET);  
write(fd, "XY", 2);
```

핵심

read는 읽은 길이만큼 위치를 옮긴다.

동작

쓰기 전 위치를 다시 지정한다.

확인

O_APPEND는 매 write를 끝에 기록한다.

실습 파일과 상세 설명은 labs/class03-02에서 확인합니다.

2부

read와 오류 처리

EOF, 짧은 읽기, 오류를 구별합니다.

read의 세 가지 결과

$n > 0$: n 바이트를 처리하고 계속 읽는다.

$n == 0$: 일반 파일의 EOF로 종료한다.

$n == -1$: `errno`를 확인한다.

이번 실습은 양수 크기로 읽는 일반 파일이다.

짧은 읽기와 EOF

10바이트 파일을 4바이트씩 읽는다.

예상 반환값은 4, 4, 2, 0이다.

2를 반환한 호출도 성공이다.

요청보다 적게 읽었다고 종료하지 않는다.

버퍼에서 유효한 범위

```
unsigned char buf[4];
ssize_t n = read(fd, buf, sizeof buf);
if (n > 0) {
    /* buf[0]부터 buf[n-1]까지 */
    use_bytes(buf, (size_t)n);
}
```

핵심

이번에 읽은 데이터는 n바이트다.

동작

read는 끝에 NUL을 붙이지 않는다.

확인

바이너리 데이터에 strlen을 쓰지 않는다.

실습 파일과 상세 설명은 labs/class03-02에서 확인합니다.

errno와 EINTR

errno는 실패 반환값을 확인한 뒤 읽는다.

EINTR: 데이터 처리 전 신호로 중단

EINTR이면 같은 요청을 다시 시도한다.

양수 반환이면 실제 처리량을 반영한다.

그 밖의 오류는 호출자에게 전달한다.

읽기 반복문의 분기

```
for (;;) {
    ssize_t n = read(fd, buf, sizeof buf);
    if (n == 0) break;
    if (n == -1) {
        if (errno == EINTR) continue;
        return -1;
    }
    /* 읽은 n바이트를 처리 */
}
```

핵심

0과 -1은 다른 종료 경로다.

동작

EINTR은 재시도한다.

확인

양수인 n만 다음 단계에 전달한다.

실습 파일과 상세 설명은 [labs/class03-02](#)에서 확인합니다.

실습 1 · 읽은 길이 관찰

```
make
RUN=$(mktemp -d ./run.XXXXXXX)
printf ABCDEFGHIJ > "$RUN/in"
./read_chunks "$RUN/in" > "$RUN/out"
cmp "$RUN/in" "$RUN/out"
echo $?
```

핵심

새 실습 폴더에서 make를 실행한다.

동작

표준 오류의 n 값을 확인한다.

확인

입력과 출력 바이트를 비교한다.

실습 파일과 상세 설명은 [labs/class03-02](#)에서 확인합니다.

3부

write_all 구현

처리한 위치와 남은 길이를 갱신합니다.

한 번의 write와 전체 기록

`write(fd, buf, 10)`이 4를 반환할 수 있다.

앞의 4바이트는 이미 기록했다.

다음 요청은 `buf + 4`에서 6바이트다.

성공한 양수 반환도 완료 여부를 확인한다.

진행 상태 계산

시작: 위치 0, 남은 길이 10

4바이트 성공: 위치 4, 남은 길이 6

2바이트 성공: 위치 6, 남은 길이 4

4바이트 성공: 위치 10, 남은 길이 0

write_all의 계약

성공: 요청한 바이트를 모두 쓰고 0 반환

실패: -1 반환, errno로 원인 전달

실패 전에 일부 바이트를 썼을 수 있다.

실습은 작은 버퍼와 blocking FD를 사용한다.

반복문과 포인터 갱신

```
const unsigned char *p = data;
size_t left = count;
while (left > 0) {
    ssize_t n = write(fd, p, left);
    /* 오류와 0 처리: 다음 슬라이드 */
    p += (size_t)n;
    left -= (size_t)n;
}
```

핵심

p는 다음 기록 위치다.

동작

left는 아직 쓰지 않은 길이다.

확인

양수 반환일 때만 두 값을 바꾼다.

실습 파일과 상세 설명은 [labs/class03-02](#)에서 확인합니다.

오류와 진행 없는 반환

```
if (n == -1) {  
    if (errno == EINTR) continue;  
    return -1;  
}  
if (n == 0) {  
    errno = EIO;  
    return -1;  
}
```

핵심

EINTR이면 위치를 유지한다.

동작

다른 오류는 -1로 전달한다.

확인

0은 EIO로 처리해 반복을 끝낸다.

실습 파일과 상세 설명은 [labs/class03-02](#)에서 확인합니다.

실습 2 · 함수 완성

```
make test-student
# 실패한 조건과 호출 인자를 확인
# student/io_helpers.h 수정 후 재실행
make test-student
```

핵심

student/io_helpers.h의 TODO를 구현한다.

동작

부분 성공 뒤 위치와 길이를 갱신한다.

확인

제공 테스트로 분기를 확인한다.

실습 파일과 상세 설명은 labs/class03-02에서 확인합니다.

반환값을 정한 테스트

가짜 write의 결과: 4, -1/EINTR, 2, 4

요청 길이는 10, 6, 6, 4가 되어야 한다.

EINTR 앞뒤의 포인터는 같아야 한다.

기록된 결과가 ABCDEFGHIJ인지 검사한다.

4부

파일 복사와 테스트

두 FD를 열고 EOF까지 복사한 뒤 결과를 검증합니다.

file_copy의 구성

입력 파일: O_RDONLY로 연다.

출력 파일: O_WRONLY | O_CREAT | O_EXCL

read로 얻은 n바이트를 write_all로 쓴다.

입력과 출력 FD를 모두 닫는다.

복사 반복문

```
for (;;) {
    ssize_t n = read(in, buf, sizeof buf);
    if (n == 0) return 0;
    if (n == -1) {
        if (errno == EINTR) continue;
        return -1;
    }
    if (write_all(out, buf, (size_t)n) < 0)
        return -1;
}
```

핵심

바깥 반복문은 EOF까지 읽는다.

동작

write_all은 n바이트를 모두 쓴다.

확인

읽기 또는 쓰기 실패는 -1로 전달한다.

실습 파일과 상세 설명은 [labs/class03-02](#)에서 확인합니다.

실패 뒤에도 FD 닫기

출력 open 실패 시 입력 FD를 닫는다.

복사 실패 시에도 두 FD를 닫는다.

close 실패도 종료 상태에 반영한다.

Linux에서 close 실패를 일괄 재시도하지 않는다.

실습 3 · 파일 복사

```
./file_copy "$RUN/in" "$RUN/copy"  
cmp "$RUN/in" "$RUN/copy"  
echo $?  
./file_copy "$RUN/in" "$RUN/copy"  
echo $?
```

핵심

새 목적지 파일 이름을 사용한다.

동작

cmp의 종료 상태를 확인한다.

확인

같은 목적지로 다시 복사해 본다.

실습 파일과 상세 설명은 [labs/class03-02](https://github.com/woowacourse/101/blob/master/lab/03/lab03-02.md)에서 확인합니다.

경계 조건과 오류 테스트

빈 파일과 NUL 포함 바이너리

버퍼 크기 4096의 앞뒤 길이

입력 없음, 기존 목적지, 동일 경로

부분 쓰기, EINTR, 0 반환, 쓰기 오류

실습 4 · 호출 관찰

```
strace -o "$RUN/trace.log" \  
-e trace=openat,read,write,close \  
./file_copy "$RUN/in" "$RUN/traced" \  
cat "$RUN/trace.log" \  
cmp "$RUN/in" "$RUN/traced"
```

핵심

Linux에서 read와 write를 추적한다.

동작

실습 파일 FD의 호출을 찾아 읽는다.

확인

반환값과 다음 요청 길이를 비교한다.

실습 파일과 상세 설명은 [labs/class03-02](#)에서 확인합니다.

코드 리뷰 · 버그 찾기

```
size_t n = read(in, buf, sizeof buf);  
if (n > 0)  
    write(out, buf, sizeof buf);
```

핵심

요청 길이와 실제 길이를 비교한다.

동작

음수 값을 unsigned로 바꾸는 시점을 본다.

확인

부분 쓰기 뒤 재요청 범위를 설명한다.

실습 파일과 상세 설명은 [labs/class03-02](#)에서 확인합니다.

정리 퀴즈

1. 10바이트 write가 4를 반환하면 다음 인자는?
2. read가 2를 반환하면 EOF인가?
3. -1 / EINTR일 때 포인터는 어떻게 바꾸는가?
4. write_all 실패 시 파일은 원래 상태인가?

제출과 다음 시간

student/io_helpers.h 구현본

REPORT.md의 예측·실행 결과·오류 분석

make test-student와 복사 비교 결과

다음 시간: FILE*와 stdio 버퍼링

참고 자료

Linux man-pages: `read(2)`, `write(2)`

Linux man-pages: `open(2)`, `close(2)`

지난 차시: 파일 프로그램 4개

실습 안내: [labs/class03-02/README.md](#)

보충

버퍼와 표준 입출력

fread와 fwrite의 크기, 반환값, 버퍼링을 살펴봅니다.

버퍼의 메모리 경계

```
char buf[4];

buf[0] = 'A'; /* 정상 */
buf[3] = 'D'; /* 정상 */

/* 다음 코드는 실행하지 않는다. */
buf[4] = 'X'; /* overflow */
buf[-1] = 'Y'; /* underwrite */
```

핵심

buf[4]에 쓰면 끝을 넘는다.

동작

buf[-1]에 쓰면 시작 앞을 침범한다.

확인

두 접근 모두 정의되지 않은 동작이다.

데이터 흐름의 underflow

버퍼는 데이터를 잠시 저장하는 공간이다.

Underflow: 소비할 데이터가 부족한 상태

예: 오디오 데이터 도착이 늦어 재생이 끊김

Overflow: 빈 공간보다 더 많은 데이터가 도착

메모리 경계 침범과 구별해 문맥을 확인한다.

read / write와 stdio

read / write: 정수 FD, 반환값은 바이트 수

fread / fwrite: FILE*, 반환값은 항목 수

stdio는 C 라이브러리 버퍼를 사용할 수 있다.

fread는 스트림에서 메모리로 읽는다.

fwrite는 메모리에서 스트림으로 쓴다.

fread와 fwrite의 인자

```
#include <stdio.h>

fread(ptr, size, nmemb, stream);
fwrite(ptr, size, nmemb, stream);

/* 요청 크기 = size * nmemb */
/* stream: FILE * */
/* 두 함수의 반환형: size_t */
```

핵심

ptr: 데이터를 담은 메모리 주소

동작

size: 항목 하나의 바이트 크기

확인

nmemb: 요청할 항목 개수

반환값의 단위

```
int values[10];
size_t n = fread(values,
                  sizeof values[0], 10, fp);
/* n == 10: int 10개를 읽음 */

unsigned char buf[100];
size_t bytes = fread(buf, 1, sizeof buf, fp);
/* bytes: 실제로 읽은 바이트 수 */
```

핵심

첫 호출의 n은 완전히 읽은 int 개수다.

동작

size가 1이면 반환값은 바이트 수다.

확인

요청 개수와 반환값을 비교한다.

fread와 버퍼 크기

```
char text[100];
size_t n = fread(text, 1,
                  sizeof text - 1, fp);
text[n] = '\0';

/* 위험한 요청: 실행하지 않는다. */
char small[10];
fread(small, 1, 100, fp);
```

핵심

최대 99바이트를 읽고 NUL을 붙인다.

동작

fread는 NUL을 자동으로 붙이지 않는다.

확인

바이너리는 읽은 n바이트로 처리한다.

fwrite와 텍스트 출력

```
int value = 1234;

size_t n = fwrite(&value,
                  sizeof value, 1, fp);
/* 성공한 항목 수 n == 1 확인 */

int rc = fprintf(text_fp, "%d\n", value);
/* 오류이면 rc < 0 */
```

핵심

fwrite는 메모리의 바이트를 기록한다.

동작

fprintf는 값을 텍스트로 변환한다.

확인

두 파일은 저장 형식이 다르다.

EOF와 오류 판별

```
size_t n = fread(buf, 1, sizeof buf, fp);
/* 먼저 읽은 n바이트를 처리 */

if (n < sizeof buf) {
    if (ferror(fp)) {
        /* 읽기 오류 처리 */
    } else if (feof(fp)) {
        /* 파일 끝 도달 */
    }
}
```

핵심

먼저 fread의 반환값을 확인한다.

동작

ferror가 참이면 읽기 오류다.

확인

feof는 읽기 후 EOF 상태를 알려 준다.

stdio 파일 복사 반복문

```
unsigned char buf[4096];
size_t n;
int failed = 0;
while ((n = fread(buf, 1, sizeof buf, src)) > 0) {
    if (fwrite(buf, 1, n, dst) != n) {
        failed = 1;
        break;
    }
}
if (ferror(src)) failed = 1;
```

핵심

읽은 n바이트만 출력한다.

동작

fwrite의 항목 수를 검사한다.

확인

읽기 오류도 실패 결과에 반영한다.

fopen과 fclose

```
FILE *src = fopen("input.bin", "rb");  
if (!src) return 1;  
FILE *dst = fopen("output.bin", "wb");  
if (!dst) { fclose(src); return 1; }  
  
/* 앞 슬라이드의 복사 반복문 삽입 */  
  
if (fclose(src) == EOF) failed = 1;  
if (fclose(dst) == EOF) failed = 1;  
return failed;
```

핵심

fopen 실패는 NULL로 확인한다.

동작

rb는 읽기, wb는 생성 또는 내용 삭제다.

확인

fclose의 출력 오류도 검사한다.

출력 버퍼와 fflush

fwrite 성공 뒤에도 출력 버퍼에 남을 수 있다.

fflush는 stdio 출력 버퍼를 커널로 내보낸다.

fflush와 fclose는 성공 0, 실패 EOF를 반환한다.

디스크 반영이 필요하면 fsync 등도 고려한다.

fseek는 스트림의 파일 위치를 변경한다.